Mining Data Stream

Ting Kai Ming
Nanjing University

Reading

- Required reading: Aggarwal Chapter 12

Acknowledgement:

Unless stated otherwise, the contents and images on the following slides are sourced from

C. C. Aggarwal, Data Mining: The Textbook. Springer, 2015.

What is data stream?

Data appears continuously in the stream, potentially infinite into the future.

No longer possible to store all the data because of resource constraints.

The streaming framework provides a viable approach, where real-time analysis can often be performed with carefully designed algorithms, without a significant investment in specialized infrastructure.

1. *Transaction streams:* data created by using a credit card, point-of-sale transaction at a supermarket, or the online purchase of an item.
2. *Web click-streams:* The activity of users at a popular Web site creates a Web click stream.
3. *Social streams:* Online social networks such as Twitter continuously generate massive text streams because of user activity.
4. *Network streams:* Communication networks contain large volumes of traffic streams.

Challenges

1. *One-pass constraint:* Because volumes of data are generated continuously and rapidly, it is assumed that the data can be processed *only once*.
2. *Concept drift:* In most applications, the data may evolve over time.
3. *Resource constraints:* The data stream is typically generated by an external process, over which a user may have very little control.
4. *Massive-domain constraints:* In some cases, when the attribute values are discrete, they may have a large number of distinct values. For example, consider a scenario, where analysis of pairwise communications in an e-mail network is desired. The number of distinct pairs of e-mail addresses in an e-mail network with 10^8 participants is of the order of 10^{16}.

Approach: Leverage synopsis

- Because of the large volume of data streams, virtually all streaming methods use an online synopsis construction approach in the mining process.
- The basic idea is to create an online synopsis that is then leveraged for mining.
- Examples of synopsis structures include random samples, bloom filters, sketches, and distinct element-counting data structures.

Synopsis Data Structures for Streams

Two types

- *Generic:* a random sample of the data points referred to as *reservoir sampling*
- *Specific:* the synopsis is designed for a specific task, such as frequent element counting or distinct element counting.

Examples of such data structures include

1. The Flajolet–Martin data structure for distinct element counting,
2. Sketches for frequent element counting or moment computation.

Reservoir Sampling

- The main advantage of sampling over other synopsis data structures is that it can be used for an arbitrary application.
- Sampling should be considered the method of choice in streaming scenarios, although it does have limitations for a small number of applications such as distinct-element counting.
- Challenge: one cannot store the entire stream on disk for sampling.
- The goal of reservoir sampling is to *continuously* maintain a *dynamically updated* sample of *k* points from a data stream without explicitly storing the stream on disk at any given point in time.

Two issues in sampling in data stream:

- Data size continually increases with time
- Previously arrived data points, which are not included in the sample, have been irrevocably lost.

Reservoir sampling algorithm

For a reservoir of size *k*, the first *k* data points in the stream are always included in the reservoir.

Subsequently, for the *n*th incoming stream data point, the following two admission control decisions are applied:

1. Insert the *n*th incoming stream data point in the reservoir with probability *k*/*n*.
2. If the newly incoming data point was inserted, then eject one of the old *k* data points in the reservoir at random to make room for the newly arriving point.

This rule maintains an unbiased reservoir sample from the data stream.

- **Lemma** *After n stream points have arrived, the probability of any stream point being included in the reservoir is the same, and is equal to k/n.*

Handling Concept Drift

Use a decay-based framework to regulate the relative importance of data points, so that more recent data points have a higher probability to be included in the sample.

Definition 12.2.1 *Let $f(r, n)$ be the bias function for the rth point at the time of arrival of the nth point. A* biased *sample $S(n)$ at the time of arrival of the nth point in the stream is defined as a sample such that the relative probability $p(r, n)$ of the rth point belonging to the sample $S(n)$ (of size n) is proportional to $f(r, n)$.*

The commonly used *exponential bias* function:

$$f(r, n) = exp[-\lambda(n-r)]$$

The *maximum reservoir requirement*: The size of the reservoir k is strictly less than $1/\lambda$.

Synopsis Structures for the Massive-Domain Scenario

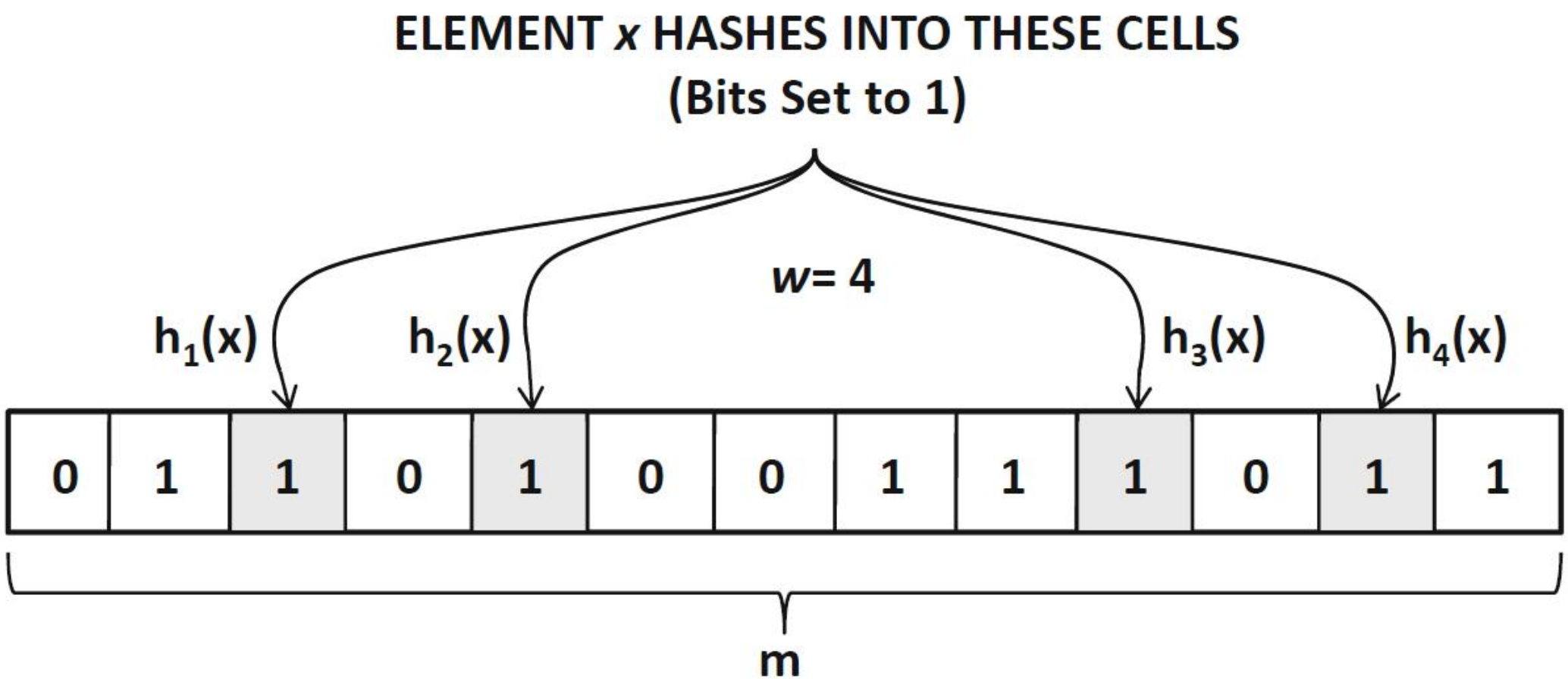
- Applications contain discrete attributes, whose domain is drawn on a large number of distinct values. Examples: IP address in a network stream, or the e-mail address in an e-mail stream.
- *Distinct element counts become more challenging from the perspective of space constraints*. For example, for an e-mail application with over a hundred million different senders and receivers, the number of possible pairwise combinations is 10^{16}.
- For many queries, such as set membership and distinct element counting, a reservoir sample would not work well.

Bloom Filter

Bloom filters are designed for set-membership queries of discrete elements.

Given a particular element, has it ever occurred in the data stream?

- One property of this data structure is that false positives are possible, but false negatives are not.
- A bloom filter is a binary bit array of length m. Thus, the space requirement of the bloom filter is $m/8$ bytes.
- The bloom filter is associated with a set of *w independent* hash functions denoted by $h_1(\cdot) \ldots h_w(\cdot)$. The argument of each of these hash functions is an element of the data stream. The hash function maps uniformly at random to an integer in the range $\{0 \ldots m - 1\}$.



MEMBERSHIP OF y (BOOLEAN RESULT) = AND { $h_1(y), h_2(y), \ldots, h_w(y)$ }

Update of bloom filter

Algorithm *BloomConstruct*(Stream: S, Size: m, Num. Hash Functions: w)
begin
Initialize all elements in a bit array B of size m to 0;
repeat
Receive next stream element $x \in S$;
for $i = 1$ to w **do**
Update $h_i(x)$th element in bit array B to 1;
until end of stream S;
return B;
end

Check for membership of y in the data stream

- The first step is to compute the hash functions $h_1(y)$ . . . $h_w(y)$. Then, it is checked whether the $h_i(y)$th element is 1.
- If at least one of the these values is 0, we are *guaranteed* that the element has not occurred in the data stream.
- if all the entries $h_1(y)$ . . . $h_w(y)$ in the bit array have a value of 1, then it is reported that y has occurred in the data stream. Applying an “AND” logical operator to all the bit entries corresponding to the indices $h_1(y)$ . . . $h_w(y)$ in the bit array.

False positive probability of Bloom Filter

- As the number of elements in the data stream increases, all elements in the bloom filter are eventually set to 1. In such a case, all set-membership queries will yield a positive response.
- Therefore, it is instructive to bound the false positive probability in terms of the size of the filter and the number of distinct elements in the data stream.
- **Lemma 12.2.2** *Consider a bloom filter B with m elements, and w different hash functions. Let n be the number of distinct elements in the stream S so far. Consider an element y, which has not appeared in the stream so far. Then, the probability F that an element y is reported as a false positive is given by the following:*

$$F = 2^{-\ln(2)m/n}$$

- *For a fixed value of the false-positive probability F, the length of the bloom filter m needs to be proportional to the number of distinct elements n in the data stream*.

Bloom Filter in practice

- Use a cascade of bloom filters for geometrically increasing values of *w*, and to use a logical AND of the membership query result over different bloom filters. This provides more stable performance over the life of the data stream.
- The bloom filter is referred to as a “filter” because it is often used to make decisions on which elements to exclude from a data stream, when they meet the membership condition.
- For example, filter forbidden elements from a universe of values, such as a set of spammer e-mail addresses in an e-mail stream. The bloom filter needs to be constructed up front with the spam e-mail addresses.
- Bloom filters are commonly used in many streaming settings in the text domain.
- Variants of Bloom Filter include count-min sketch

Count-Min Sketch

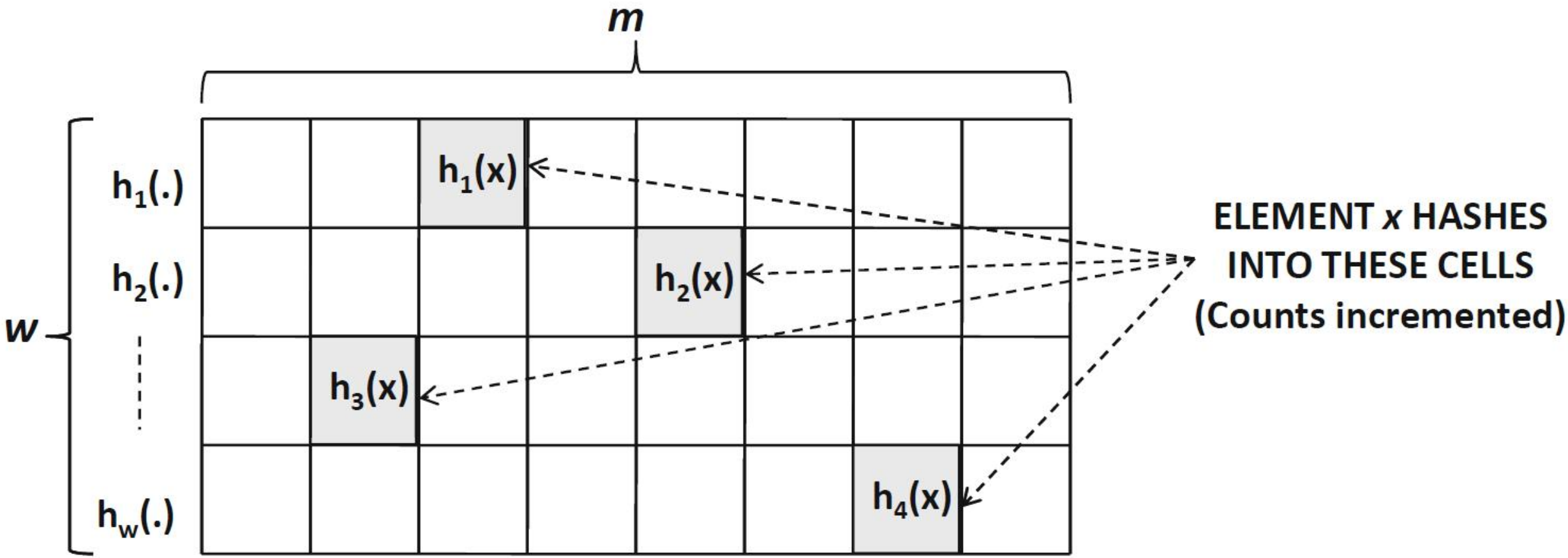
The count-min sketch is designed for *count-based* summaries.

The count-min sketch can be viewed as a $w \times m$ 2-dimensional array of cells. Each of the w numeric arrays corresponds to a hash function.

1. The ith hash function $h_i(\cdot)$ maps a stream element to an integer in the range $[0 \ldots m-1]$. This mapping can also be viewed as one of the index values in the ith numeric array.
2. The w hash functions $h_1(\cdot) \ldots h_w(\cdot)$ are fully independent of one another, but *pairwise* independent over different arguments. In other words, for any two values x_1 and x_2, $h_i(x_1)$ and $h_i(x_2)$ are independent.

For each incoming stream element x, the element $h_i(x)$ is incremented by 1

Example array of Count-Min Sketch



ESTIMATED COUNT OF x = min { $h_1(x)$, $h_2(x)$, …, $h_w(x)$ }

Determine frequency of an element from Count-Min Sketch

- The count-min sketch can be used at any time during the progression of the stream.
- To determine the frequency of an element y, the value $V_i(y)$ of the $(i, h_i(y))$th array element is retrieved.
- The tightest possible estimate may be obtained by using the minimum value $\min_i\{V_i(y)\}$ over the different hash functions.
- Note that the count-min sketch causes an overestimation of frequency values because of collisions of nonnegative frequency counts of distinct stream items. It is upper bounded by $E(y) \leq G(y) + (n_f \cdot e)/m$

where $G(y)$ is true frequency of item y; n_f is the total frequencies of all items received so far; e represents the base of the natural logarithm; a count-min sketch of size $w \times m$

Applications of count-min sketch

- Estimating the join size on the massive-domain attribute in two data streams: The dot product of their corresponding count-min arrays for each (same) hash function is computed. The minimum value of the dot product over the *w* different arrays is reported as the estimation.
- The determination of quantiles.
- Identify the frequent elements, also referred to as *heavy hitters*.

Alon–Matias–Szegedy (AMS) sketch

- Use to query the second-order moment F_2 of the data stream with *n distinct* elements

$$F_2 = \sum_{i=1}^{n} f_i^2$$

- In the massive-domain scenario, where the number of distinct elements is large, this quantity is hard to estimate because running counts of the frequencies f_i cannot be maintained with an array.
- F_2 can be estimated efficiently using the AMS sketch

AMS sketch has *m* different sketch components

- Each sketch component is associated with an independent hash function
- A random binary value $r \in \{-1, 1\}$ is generated for the incoming stream element using a hash function for a component.
- The component Q of the sketch, for a stream of n distinct elements with aggregate frequencies $f_1 \ldots f_n$, can be shown to be equal to

$$Q = \sum_{i=1} f_i \cdot r_i$$

- **Lemma 12.2.4** *The second moment of the data stream can be estimated by the square of the AMS sketch component Q:*

$$F_2 = E[Q^2]$$

- Use the AMS sketch to estimate frequency values. For the *j*th distinct stream element with frequency f_j, the product of the random variable r_j and Q_i provides an estimate of the frequency.

Flajolet–Martin Algorithm for Distinct Element Counting

- Distinct element counting is influenced by the much larger number of infrequent items in a data stream.
- The Flajolet–Martin algorithm uses a hash function $h(\cdot)$ to render a mapping from a given element x in the data stream to an integer in the range $[0, 2^L - 1]$.
- The position R of the rightmost 1 bit of the binary representation of the integer $h(x)$ is determined. The value of R represents the number of trailing zeros in this binary representation.
- Let *Rmax* be the maximum value of R over all stream elements. The value of *Rmax* can be maintained incrementally in the streaming scenario by applying the hash function to each incoming stream element, determining its rightmost bit, and then updating *Rmax*.

Key idea

- The key idea in the Flajolet–Martin algorithm is that the dynamically maintained value of *Rmax* is logarithmically related to the number of distinct elements encountered so far in the stream:

$$E[Rmax] = \log_2(\phi n), \quad \phi = 0.77351.$$

- The value of $2^{Rmax}/\phi$ provides an estimate for the number of distinct elements n.

Note: the bloom filter can also be used to estimate the number of distinct elements. However, the bloom filter is not a space-efficient way to count the number of distinct elements when set-membership queries are not required.

Summary on sketches

Bloom filter	Set membership
min-count sketch	Frequent item count
AMS sketch	Second-order moment query
Flajolet–Martin algorithm	Distinct element counting

Discussion

1. *Explain what concept drift is. How does it relate to iid (independent and identically distributed)?*
2. *Describe the difference between data stream and timeseries. How does concept drift in a data stream relate to that in timeseries?*
3. *Out of the four constraints in data streams, explain which constraints the synopsis approach addresses.*

Challenges

1. *One-pass constraint:* Because volumes of data are generated continuously and rapidly, it is assumed that the data can be processed *only once*.
2. *Concept drift:* In most applications, the data may evolve over time.
3. *Resource constraints:* The data stream is typically generated by an external process, over which a user may have very little control.
4. *Massive-domain constraints:* In some cases, when the attribute values are discrete, they may have a large number of distinct values. For example, consider a scenario, where analysis of pairwise communications in an e-mail network is desired. The number of distinct pairs of e-mail addresses in an e-mail network with 10^8 participants is of the order of 10^{16}.

Clustering Data Streams

- The problem of clustering is especially significant in the data stream scenario because of its ability to provide a compact synopsis of the data stream.
- A clustering of the data stream can often be used as a heuristic substitute for reservoir sampling, especially if a fine-grained clustering is used. For these reasons, stream clustering is often used as a precursor to other applications such as streaming classification.

CluStream Algorithm

- Apply the clustering process with a two-stage methodology, including an online microclustering stage, and an offline macroclustering stage.
- The online microclustering stage processes the stream in real time to continuously maintain summarized but detailed cluster statistics of the stream. These are referred to as *microclusters*.
- The offline macroclustering stage further summarizes these detailed clusters to provide the user with a more concise understanding of the clusters over different time horizons and levels of temporal granularity.

Microcluster Definition

Definition 12.4.1 *A microcluster for a set of d-dimensional points $X_{i_1} \ldots X_{i_n}$ with time stamps $T_{i_1} \ldots T_{i_n}$ is the $(2 \cdot d + 3)$ tuple $(\overline{CF2^x}, \overline{CF1^x}, CF2^t, CF1^t, n)$, wherein $\overline{CF2^x}$ and $\overline{CF1^x}$ each correspond to a vector of d entries. The definition of each of these entries is as follows:*

1. *For each dimension, the sum of the squares of the data values is maintained in $\overline{CF2^x}$. Thus, $\overline{CF2^x}$ contains d values. The p-th entry of $\overline{CF2^x}$ is equal to $\sum_{j=1}^{n} (x_{i_j}^p)^2$.*

2. *For each dimension, the sum of the data values is maintained in $\overline{CF1^x}$. Thus, $\overline{CF1^x}$ contains d values. The p-th entry of $\overline{CF1^x}$ is equal to $\sum_{j=1}^{n} x_{i_j}^p$.*

3. *The sum of the squares of the time stamps $T_{i_1} \ldots T_{i_n}$ is maintained in $CF2^t$.*

4. *The sum of the time stamps $T_{i_1} \ldots T_{i_n}$ is maintained in $CF1^t$.*

5. *The number of data points is maintained in n.*

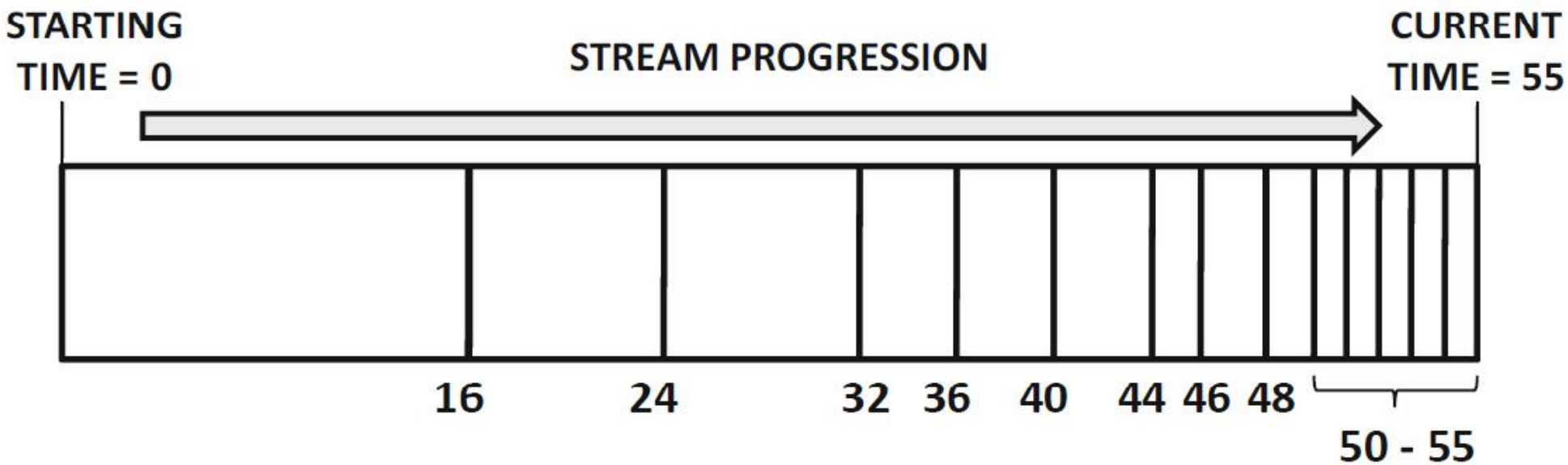
Microclustering Algorithm

Whenever a new data point X_i arrives, the microclusters are updated to reflect the changes. Each data point either needs to be absorbed by a microcluster, or it needs to be put in a cluster of its own.

- The distance of the data point to the current microcluster centroids $M_1 \ldots M_k$ is determined.
- The data point X_i is assigned to its closest cluster M_p, unless it is deemed that the data point does not "naturally" belong to that (or any other) microcluster.
- If the data point does not lie within the maximum boundary of the nearest microcluster, then a new microcluster must be created containing the data point X_i.
- This can be achieved by either deleting an old microcluster or merging two of the older clusters. This decision is made by examining the staleness of the different clusters, and the number of points in them.

Pyramidal Time Frame

Order of Snapshots	Clock times (last five snapshots)
0	55 54 53 52 51
1	54 52 50 48 46
2	52 48 44 40 36
3	48 40 32 24 16
4	48 32 16
5	32



Observations at any moment in time over the course of the data stream

- The maximum order of any snapshot stored at T time units since the beginning of the stream mining process is $\log_\alpha(T)$.
- The maximum number of snapshots maintained at T time units since the beginning of the stream mining process is $(\alpha^l + 1) \cdot \log_\alpha(T)$, where l *regulates the number of snapshots at each order*
- For any user-specified time horizon h, at least one stored snapshot can be found.
- Example: For $l = 10$, it is possible to approximate any time horizon within 0.2 %. At the same time, a total of only $(2^{10} + 1) \cdot \log_2(100 * 365 * 24 * 60 * 60) \approx 32,343$ snapshots are required for a stream with a clock granularity of 1 s and running over 100 years.

Massive-Domain Stream Clustering: *CSketch*

- The idea in this method is to use a count-min sketch to store the frequencies of attribute–value combinations in each cluster. Thus, the number of count-min sketches used is equal to the number of clusters.
- An online *k*-means style clustering is applied, in which the sketch is used as the representative for the (discrete) attributes in the cluster.
- For any incoming data point, a dot product is computed with respect to each cluster.
- The data point is assigned to the cluster with which it has the largest dot product. The statistics in the sketch representing that particular cluster are then updated.
- Unlike microclustering, it does not implement the merging and removal steps.

Streaming Classification

- Decision Trees approach
- Supervised Microcluster Approach
- Massive-Domain Streaming Classification
- The SENC problem

VFDT (Very Fast Decision Trees)

- VFDT assumes that the concept does not change during the data stream. With this assumption, it is able to build a tree incrementally as data arrive; and the tree is as good as that built in batch mode using a dataset of the same size, seen in the data stream. It does not deal with concept change.
- In the context of data streams with continuously accumulating samples, the key in VFDT is to *wait* for a large enough sample size *n* before performing the split. The Hoeffding tree approach determines whether the current difference e in the Gini index between the best and second-best split attributes is at least sqrt[$R^2 \cdot \ln(1/\delta)/2n$] in order to initiate a split. This provides a guarantee on the quality of a split at a particular node.
- *R* denotes the range of the split criterion: For the Gini index, the value of *R* is 1. e is the difference (or error) in Gini indexes will not occur with at least probability 1-δ.

The *CVFDT* approach

CVFDT incorporates two main ideas to address the additional challenges of drift:

1. A sliding window of training items is used to limit the impact of historical behavior.
2. Alternate subtrees at each internal node *i* are constructed to account for the fact that the best split attribute may no longer remain the top choice because of stream evolution.

Supervised Microcluster Approach

- In the streaming scenario, it is difficult to efficiently compute the *k* nearest neighbors for a particular test instance because of the increasing size of the stream.
- A supervised variant of microclustering is used in which data points of different classes are not allowed to mix within clusters. Labels are associated with microclusters rather than individual data points.
- The dominant label of the top-*k* nearest microclusters is reported as the relevant label.
- To deal with concept drift, a part of the training stream is separated out as the validation stream. Recent parts of the validation stream are utilized as test cases to evaluate the accuracy over different time horizons. The optimal horizon is selected. The *k*-nearest neighbor approach is applied to test instances over this optimally selected horizon.

Massive-Domain Streaming Classification

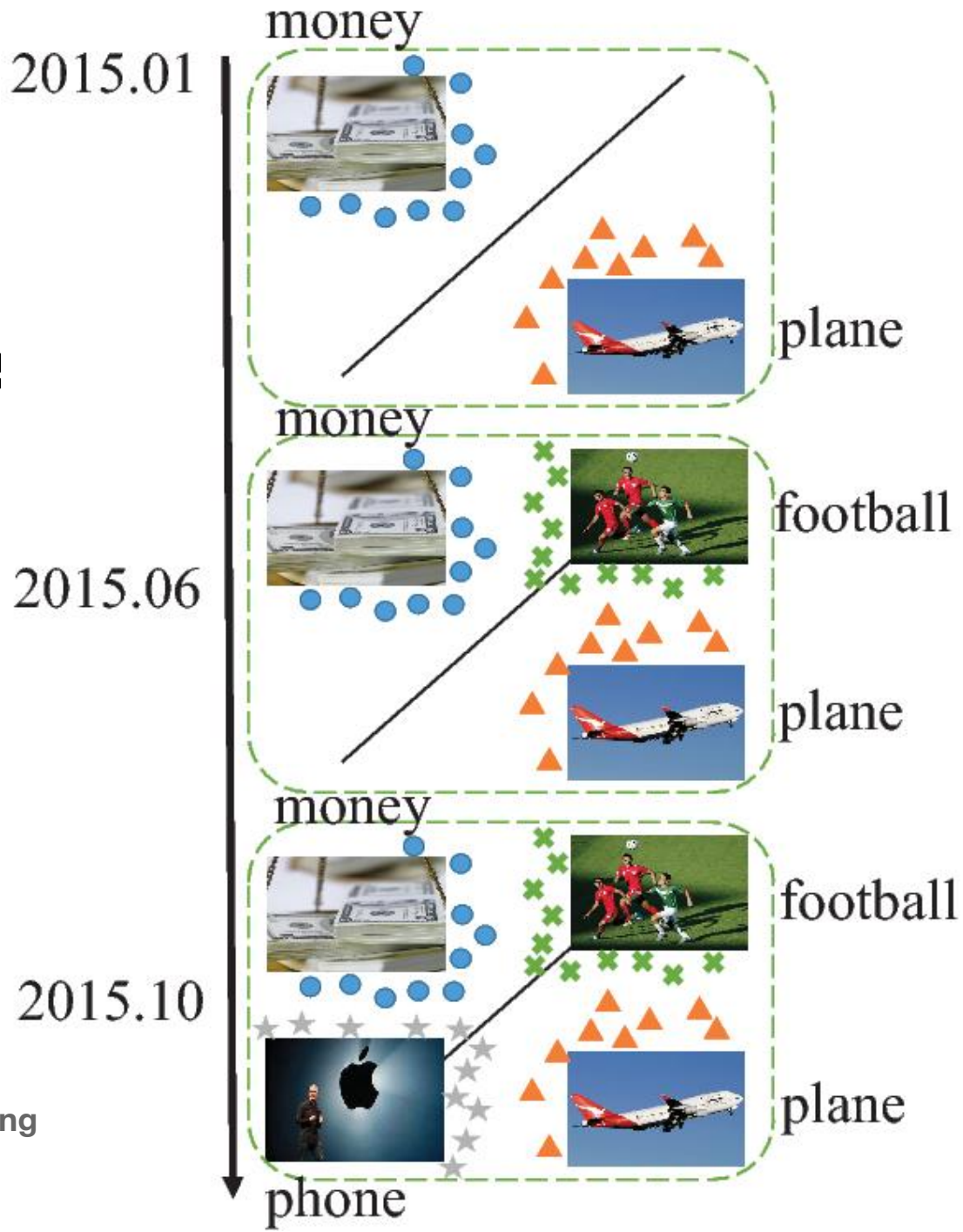
- The count-min sketch is used based on a rule based approach.
- Each class is associated with a sketch that is used to track frequent r-combinations of items in the training data, where r is bounded above by a small number k. For each incoming training data point, all possible r-combinations (for $r \leq k$) are treated as pseudo-items that are added to the sketch of the relevant class.
- Different classes will have different relevant pseudo-items that will show up in the varying frequencies of the cells belonging to sketches of different classes.
- The frequent discriminative pseudo-items are determined to create *implicit* rules relating the pseudo-items to the different classes.
- For a given test instance, the rule is determined, which pseudo-items correspond to the combination of items inside them. The discriminative ones among them are determined by retrieving their statistics from the class-specific sketches.
- These are then used to perform the classification of the test instance, using the same general approach as a rule-based classifier.

Discussion

1. Explain how one can use microclusters to perform clustering.
2. Specify whether k-means, k-medoids or DBSCAN can use microclusters to perform clustering

The SENC problem

- SENC: Classification unde streaming emerging new classes:



Xin Mu, Kai Ming Ting, Zhi-Hua Zhou:
Classification Under Streaming Emerging New Classes: A Solution Using Completely-Random Trees. IEEE Trans. Knowl. Data Eng. 29(8): 1605-1618 (2017)

Three subproblems in SENC

1. Detecting emerging new classes
2. Classifying known classes
3. Updating models to integrate each new class as part of the known classes

Existing common approach

- Treats the SENC problem as a classification problem i.e., focus on the second and the third subproblems.
- Solves it using either a supervised learner or a semi-supervised learner.
- Uses a clustering method to identify new classes.

Overall Aim of the SENC task

- Maintain high classification accuracy continuously in a data stream.
- Challenges: (i) detect emerging new classes and classify instances of known classes with high accuracy; and (ii) perform model update efficiently and maintain model complexity in a reasonable size.
- Constraints: The hardest condition: True labels are not available through the entire stream except the initial training set; no training data is stored and no complete retraining.

Distinguishing features of SENCForest

- Employ **unsupervised anomaly detector** as the basis to solve the problem, and have **a common core** which acts as distinct unsupervised learner and supervised learner.
- Use **completely-random trees** to construct the common core which acts as the detector as well as the classifier.
- Model is updated without the initial training set.

Main contribution

- Treat the emerging new class detection problem as an anomaly detection problem.
- The proposal shifts the focus of treating SENC as a classification problem to one based on unsupervised anomaly detection problem.
- In other words, the focus is shifted from the second subproblem to the first subproblem which is more critical in solving the entire problem.
- This shift brings about an integrated approach to solve all three subproblems in SENC with a **common core**.

Ideas

1. Intuition: instances of **emerging new classes are 'outlying anomalies'** wrt the instances of known classes. The assumption is that anomalies of the known classes are more "normal" than the "outlying" anomalies.
2. **Completely-random trees** have been shown to be competitive classifier and anomaly detector independently. This provides an opportunity to use the trees **as a common core** to solve all three subproblems in SENC.

The proposed method: SENCForest

- SENCForest is a modified version of iForest (Liu at al, ICDM 2008) which has the ability to detect emerging new classes as ‘outlying anomalies’.
- It is also used as a classifier

Discussion

6. To deal with concept drift, a fixed length sliding window is often used. Discuss the merit and limitation of this approach.
7. Discuss how you may use a sketch for anomaly detection in data streams.
8. Discuss the disadvantage and advantage of using a sketch for anomaly detection.
9. Explain the nature of the SENC (Classification under streaming emerging new classes) problem: is it a classification problem only? Explain your answer.

Discussion

10. Clustering is sometimes suggested as a method for outlier detection. Discuss the advantage and disadvantage of this method in comparison to outlier detection methods such as LOF (Local Outlier Factor) and Isolation Forest.

11. Discuss the generality of different synopsis construction methods to various stream mining problems. Why is it difficult to apply these methods to outlier analysis?

Discussion

12. The problem of streaming classification is especially challenging because of the impact of concept drift. One simple approach is to use a reservoir sample to create a concise representation of the training data. This concise representation can be used to create an offline model. If desired, a decay-based reservoir sample can be used to handle concept drift. Such an approach has the advantage that any conventional classification algorithm can be used since the challenges associated with the streaming paradigm have already been addressed at the sampling stage.

    Discuss the issues in using this approach.

Comment from the textbook

“… drift detection and recovery has been thoroughly investigated for streaming data where labels are immediately (and fully) available. In contrast, not as much research has been conducted on the efficiency of drift detection methods for streaming data where labels arrive with a non-negligible delay or when some (or all) labels never arrive …”

Reflection

- What have you learned this week?
  (Every student shall attempt to answer this question.)

Recent publications on streaming data mining

- Yang Cao, Ye Zhu, Kai Ming Ting, Flora D. Salim, Hong Xian Li, Luxing Yang, Gang Li: Detecting Change Intervals with Isolation Distributional Kernel. Journal of Artificial Intelligence Research 79: 273-306, 2024.
- Xin Han, Ye Zhu, Kai Ming Ting, De-Chuan Zhan, Gang Li: Streaming Hierarchical Clustering Based on Point-Set Kernel. KDD 2022: 525-533
- Xin-Qiang Cai, Peng Zhao, Kai Ming Ting, Xin Mu, Yuan Jiang. Nearest Neighbor Ensembles: An Effective Method for Difficult Problems in Streaming Classification with Emerging New Classes. In: Proceedings of the 19th IEEE International Conference on Data Mining, 2019. Page: 970-975.